LECTURE 3 · A STORY IN FOUR CHAPTERS

Accessible Component Systems, WCAG 2.1 AA & AI-Generated UI Verification

How the web agreed everyone gets to use it — and what to check when an AI forgets.

Instructor Mainack Mondal
Dept. of Computer Science and Engineering, IIT Kharagpur
mainack@cse.iitkgp.ac.in

IIT Kharagpur × upGrad

Today's Story, in Four Chapters

Ch.1	Component architecture — Tailwind CSS and shadcn/ui
Ch.2	WCAG 2.1 AA — keyboard navigation and ARIA semantics
Ch.3	axe-core — automated accessibility testing
Ch.4	AI-generated UI verification

By the final page, you'll be able to...

Build an accessible component system that passes a WCAG 2.1 AA audit — AI-written code included.

CHAPTER 1

Component Architecture with Tailwind CSS & shadcn/ui

A developer got tired of naming CSS classes. That's the whole origin story.

First, Refresher: What Is CSS?

- 1996 — CSS1 separates visual style from HTML content
- One stylesheet controls color, spacing, layout for a whole site
- 2011 — CSS3 modules add flexbox, grid, animation
- Powerful — but every team invents its own class-naming system

What the Naming Chaos Actually Looked Like

A real pattern.

styles.css

```
/* 2018 — original card header */
.card-header { padding: 16px; font-weight: 600; }

/* 2019 — redesign; old class kept, just in case */
.card-header-v2 { padding: 12px 16px; font-weight: 700; }

/* 2021 — another redesign, another new name */
.card-header-new { padding: 14px; font-weight: 600; }

/* 2022 — one dev tries BEM, nobody else follows */
.card-header--compact { padding: 8px; }
```

Nobody deletes anything

No one can tell which class is still referenced somewhere in the app, so all four ship forever.

Four names, one purpose

Every version styles the same visual element — the only thing that changed is what someone decided to call it.

> *This is the pattern that made Adam Wathan ask: what if we just stopped naming things?*

How Utility-First CSS Came to Be

- CSS files only ever grew — nobody dared delete a class
- .card-header became .card-header-v2 rather than risk breaking things
- 2017 — Adam Wathan ships Tailwind CSS
- The idea: stop naming things, compose utilities instead
- Utility-first becomes the industry default

> *The unfashionable idea won.*

The Same Header, the Utility-First Way

No new class name needed — style composed directly where it's used.

card-header.tsx

```tsx
function CardHeader({ children }) {
  return (
    <div className="p-4 font-semibold">
      {children}
    </div>
  );
}

// Need it more compact somewhere else?
// Compose different utilities — no new class to invent,
// nothing old left behind to wonder about.
<div className="p-2 font-semibold">...</div>
```

> *Four stylesheet entries from the last slide collapse into zero naming decisions on this one.*

From Utility Classes to Owned Components

- 2017 — Tailwind CSS: styling becomes composition
- 2020 — Radix UI: unstyled, accessible primitives
- 2022 — shadcn/ui: Radix's accessibility + Tailwind's styling, copied into your repo
- Today — the default start for new Next.js + Tailwind apps

Each layer fixed the last layer's leftover problem.

Three Pillars of Component Architecture

Tailwind CSS

- Utility-first class engine
- Purged at build time
- Constraints via config
- Zero runtime CSS cost

shadcn/ui

- Copy-paste, not npm install
- Built on Radix primitives
- You own the code
- Accessible by default

Design Tokens

- Color, spacing, radius scales
- CSS variables for theming
- Dark mode = token swap
- One visual source of truth

Ownership beats abstraction — shadcn/ui lives in your repo, not node_modules.

What shadcn/ui Actually Brings to the Table

Same Button component, two distribution models — look at where the code actually lives.

VS Code — Explorer

Traditional library (MUI, Chakra)

```
your-app/
  node_modules/
    @mui/material/  (thousands of files)
      Button/Button.js  <- not yours
  src/
    App.tsx
      import { Button } from '@mui/material'
```

shadcn/ui

```
your-app/
  components/
    ui/
      button.tsx   <- yours, editable
      dialog.tsx   <- yours, editable
      card.tsx     <- yours, editable
  lib/utils.ts
```

There is no node_modules/shadcn-ui to update or fight with — after the CLI runs, the component is just files in your project.

Radix's Design Principles, Spelled Out

Straight from Radix's own docs — this is what shadcn/ui inherits underneath its Tailwind styling.

Accessible

- Follows WAI-ARIA authoring patterns
- Handles aria/role attributes internally
- Manages focus and keyboard nav for you

Unstyled

- Ships with zero CSS, by design
- Works with any styling solution
- Tailwind is a choice, not a requirement

Composable & Open

- Granular access to every component part
- Uncontrolled by default, controllable when needed
- Adopt one primitive, or the whole system

shadcn/ui didn't invent accessible behavior — it packaged Radix's principles with Tailwind's styling model and made both editable.

Why AI Coding Agents Love shadcn/ui

Ownership isn't just a philosophy — it's what makes an AI agent's suggestions actually usable.

- An AI agent can only fix code it can see and edit
- npm packages hide their internals three layers deep
- shadcn/ui components sit as plain files in your repo
- The AI's next suggestion edits real source, not a black box
- This is why most AI-native starter kits ship with shadcn/ui

Copy-paste distribution turns out to be an AI-tooling advantage nobody planned for in 2022.

The Prompt, in Practice

What actually happens when this chapter's ideas meet a real AI coding agent.

prompt to Claude Code

```
Add a Tooltip component to components/ui/ following
this project's shadcn/ui conventions.

- Dark background, compact padding
- Should work with any trigger element
- Match the variant pattern already used in button.tsx
```

Pros

Correctly scaffolds using Radix's Tooltip primitive, matches the existing cva variant pattern from button.tsx.

Cons

Hardcoded a hex color instead of referencing the design token — caught on review, not by the AI itself.

Good output, one real mistake — this is why 'verify' stays in the loop even for a chapter this well-trodden.

Button Component with Variants (cva)

One component, many states — class-variance-authority maps props to classes.

components/ui/button.tsx

```tsx
import { cva, type VariantProps } from "class-variance-authority";

const buttonVariants = cva(
  "inline-flex items-center justify-center rounded-md " +
  "font-medium transition-colors focus-visible:outline-none " +
  "focus-visible:ring-2 disabled:opacity-50",
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground",
        outline: "border border-input bg-background",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        destructive: "bg-destructive text-white",
      },
      size: { sm: "h-8 px-3 text-sm", md: "h-10 px-4", lg: "h-12 px-6 text-lg" },
    },
    defaultVariants: { variant: "default", size: "md" },
  }
);
```

cva()

Maps props to a class string. No runtime CSS cost.

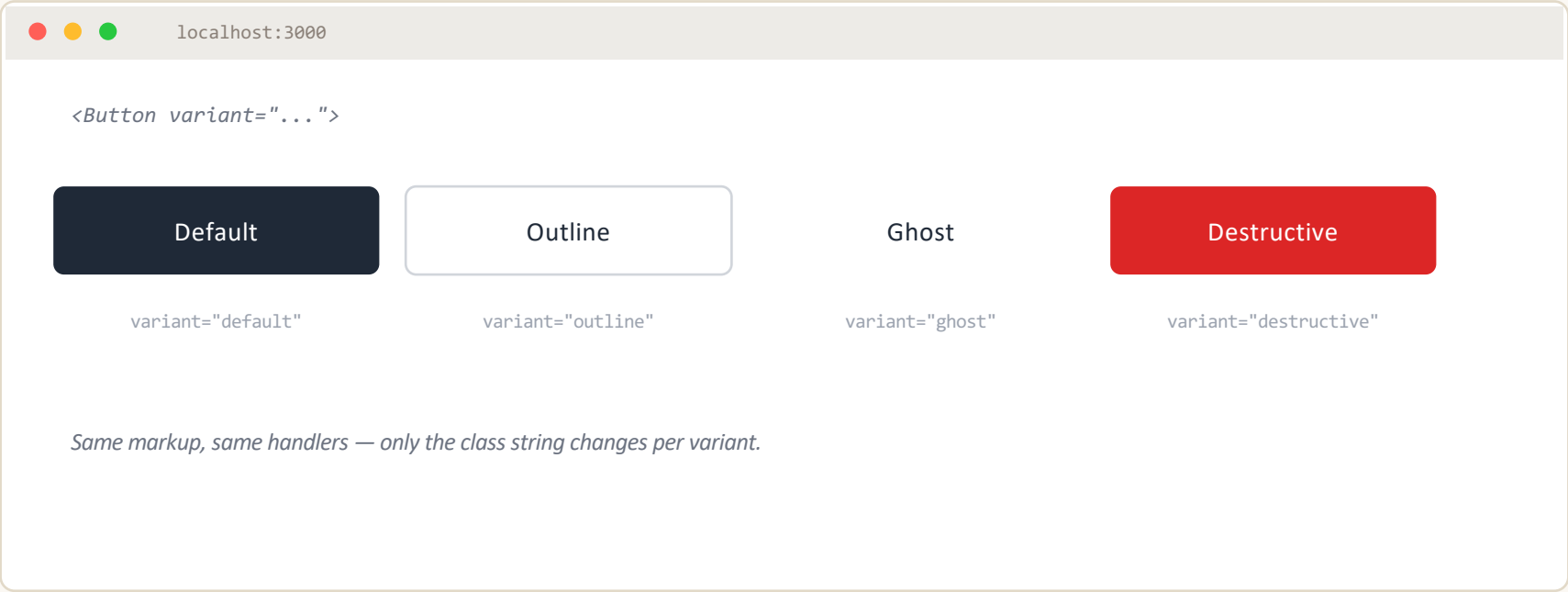
Type safety

Invalid variant values fail at compile time.

Small API, every visual state covered.

What the Button Variants Actually Look Like

Four variant props from the code on the last slide, rendered.



Default

Outline

Ghost

Destructive

variant="default"

variant="outline"

variant="ghost"

variant="destructive"

Same markup, same handlers — only the class string changes per variant.

One component definition, four production-ready looks.

The Compound Component Pattern

One <Card> shouldn't need forty props for every header/body/footer combo.

The Problem

- Monolithic components collect dozens of props
- Names collide across variants
- Markup order becomes rigid

The Fix

- Parent provides context (<Card>)
- Children consume it (<CardHeader>)
- Composition replaces prop explosion
- Consumer controls the order

This is how Dialog, Select, and Tabs stay flexible without a 40-prop API.

Compound Components in Practice

app/dashboard/page.tsx

```
<Card>
  <CardHeader>
    <CardTitle>Monthly Revenue</CardTitle>
    <CardDescription>Compared to last month</CardDescription>
  </CardHeader>

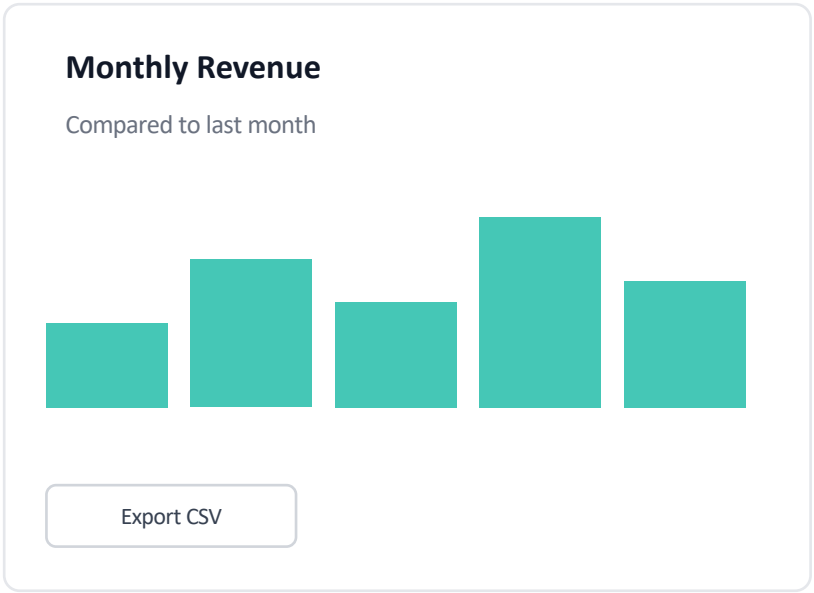
  <CardContent>
    <RevenueChart data={revenueData} />
  </CardContent>

  <CardFooter>
    <Button variant="outline">Export CSV</Button>
  </CardFooter>
</Card>
```

No titleText or footerContent props — reorder freely.

What the Rendered Card Looks Like

localhost:3000/dashboard



<CardHeader> / <CardContent> / <CardFooter> map directly to the sections you see here.

The compound structure in code is literally the visual structure on screen.

shadcn/ui vs a Full Component Library

? Need pixel-level control?	→ shadcn/ui
? Need a large suite fast?	→ MUI / Chakra
? Team fluent in Tailwind?	→ shadcn/ui
? AI tools will edit internals?	→ shadcn/ui

Chapter 1 Key Terms

Background material worth keeping — five terms that recur for the rest of the course.

Terms

- Utility-first CSS — styling by composing small classes, not naming custom ones
- cva (class-variance-authority) — maps component props to a class string
- Compound component — a parent + children that share implicit context
- Design token — a named value (color, spacing) used instead of a raw number
- Radix UI — the unstyled, accessible primitives shadcn/ui is built on

END OF CHAPTER 1

Component Architecture

- ✓ Tailwind: styling by composition, not naming
- ✓ shadcn/ui: own the code, don't depend on it
- ✓ Compound components replace prop explosion
- ✓ AI agents edit shadcn/ui's real source — never a black box
- ✓ Design tokens keep theming consistent everywhere

The takeaway: *Own your components — copy-paste beats a black-box dependency.*

Today's Story, in Four Chapters

✓	~~Component architecture — Tailwind CSS and shadcn/ui~~
Ch.2	WCAG 2.1 AA — keyboard navigation and ARIA semantics
Ch.3	axe-core — automated accessibility testing
Ch.4	AI-generated UI verification

Up next

WCAG 2.1 AA — the rulebook the web quietly agreed to follow, and why keyboard and ARIA support aren't optional.

Chapter 1: Poll

CHAPTER 2

WCAG 2.1 AA: Keyboard Navigation & ARIA Semantics

In 1997, the web's inventor decided it needed a promise. This is that promise.

What Is WCAG, and Why 'AA'?

Before the history — the two acronyms this entire chapter hangs on.

WCAG, Spelled Out

- Web Content Accessibility Guidelines
- Written and maintained by the W3C
- A testable rulebook, not a suggestion — every rule has a pass/fail

Why 'AA' Specifically

- Three levels exist: A, AA, AAA
- AA is the level referenced by law and industry worldwide
- It's the bar this course targets — rigorous, but shippable

Everything in this chapter is really just: what does it take to satisfy 'AA'?

Why This Isn't Optional

The goal, stated plainly: real people are locked out, and regulators have noticed.

1.3B

people worldwide live with a significant disability — WHO, 16% of the global population

4,000+

digital accessibility lawsuits filed in US courts in 2024 alone (UsableNet)

3

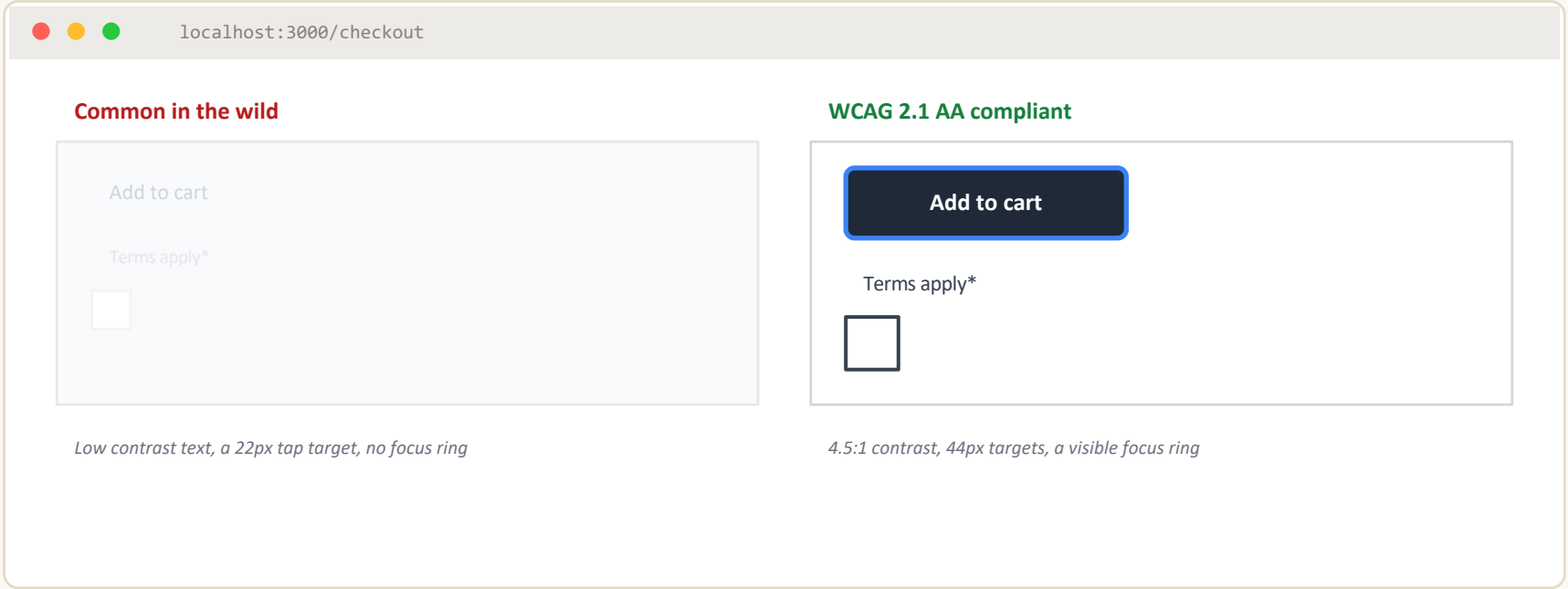
major regulations now in force: US ADA, EU Accessibility Act (June 2025), India's RPWD Act

India's RPWD Act 2016 (Section 42) points directly to WCAG 2.1 via the IS 17802 standard — this is not just a US problem.

Two stakeholders care: the users who are locked out, and the regulators who now fine you for it.

What 'Inaccessible' Actually Looks Like

Same feature, two implementations — only one of them is usable by everyone.



Both compile. Both ship. Only one can be used by someone with low vision or a motor impairment.

Remember Radix, From Chapter 1?

A quick callback before we go further — this is where shadcn/ui's foundation pays off.

- Chapter 1: shadcn/ui is built on Radix UI primitives
- Radix UI, in full: a library of unstyled, accessible component behavior
- Radix ships the keyboard logic and ARIA wiring — pre-solved
- shadcn/ui adds Tailwind styling on top; the accessibility underneath is Radix's
- This is why 'the Dialog already does this via Radix' keeps coming up

Everything in this chapter, shadcn/ui's defaults already attempt — your job is to know it well enough to verify that, not take it on faith.

How WCAG Came to Be

- 1997 — Tim Berners-Lee launches the Web Accessibility Initiative
- 1999 — WCAG 1.0: the first testable accessibility standard
- 2008 — WCAG 2.0 introduces the POUR principles
- 2018 — WCAG 2.1 adds mobile, touch, cognitive criteria
- Today — AA is the practical, legally-referenced bar

Rewritten every decade to match how we actually use the web.

The Four Principles: POUR

Perceivable

- Text alternatives for images
- Sufficient color contrast
- Content adapts without losing meaning

Operable

- Fully keyboard operable
- No seizure-inducing content
- Enough time to read and act

Understandable / Robust

- Predictable navigation
- Errors identified clearly
- Works with assistive tech

Testable, not aspirational — every criterion has a pass/fail.

Conformance Levels: A, AA, AAA

Level A

- Bronze — minimum bar
- e.g. alt text
- Rarely enough alone

Level AA — Our Target

- The legal standard
- 4.5:1 contrast minimum
- Full keyboard support

Level AAA

- Gold — often impractical
- 7:1 contrast
- Applied selectively

AA balances rigor with what's actually shippable.

Keyboard Navigation Requirements

Unplug your mouse. If a task breaks, WCAG says the interface is broken.

Must Work

- Logical tab order
- Visible focus indicator
- Enter/Space activates
- Escape closes dialogs

Must Never Happen

- Keyboard traps
- Positive tabindex values
- Missing arrow-key support
- outline: none, no replacement

Five keys: Tab, Shift+Tab, Enter, Space, Escape.

What 'Full' in Full Keyboard Support Actually Means

'Full' is doing a lot of work in that phrase — here's exactly what it covers, and what it doesn't.

'Full' Means

- Every interactive element is reachable, in order
- Every custom widget's states are operable, not just buttons
- No dead ends — you can always Tab or Escape your way out

Keyboard Access Alone Does NOT Give You

- An accessible name a screen reader can announce
- A sensible reading order for non-visual users
- Meaning conveyed without relying on color or position alone

A button can be perfectly keyboard-operable and still be invisible to a screen reader — that gap is exactly what ARIA exists to close, next.

Focus Trap for a Modal Dialog

Keeps Tab cycling inside the dialog while it's open.

hooks/useFocusTrap.ts

```
function useFocusTrap(ref: RefObject<HTMLElement>) {
  useEffect(() => {
    const node = ref.current;
    if (!node) return;
    const focusables = node.querySelectorAll(
      'button, [href], input, [tabindex]:not([tabindex="-1"])'
    );
    const first = focusables[0] as HTMLElement;
    const last = focusables[focusables.length - 1] as HTMLElement;

    function onKeyDown(e: KeyboardEvent) {
      if (e.key !== "Tab") return;
      if (e.shiftKey && document.activeElement === first) {
        e.preventDefault(); last.focus();
      } else if (!e.shiftKey && document.activeElement === last) {
        e.preventDefault(); first.focus();
      }
    }
    node.addEventListener("keydown", onKeyDown);
    first?.focus();
    return () => node.removeEventListener("keydown", onKeyDown);
  }, [ref]);
}
```

> *shadcn/ui's Dialog already does this via Radix.*

Why ARIA Had to Be Invented

- Mid-2000s — AJAX lets pages update without a reload
- Screen readers had no idea content had changed
- 2005-2014 — W3C standardizes WAI-ARIA
- ARIA describes state changes to assistive tech
- It changes nothing visually — only what's announced

ARIA is a translation layer for what a screen can't watch change.

ARIA's Full Name, and How It Actually Works

ARIA = Accessible Rich Internet Applications, part of the broader WAI-ARIA specification.

- Your JSX renders into the browser's DOM, as usual
- The browser builds a second, parallel structure: the accessibility tree
- ARIA attributes (role, aria-*) shape what goes into that tree
- The OS accessibility API (e.g. UIA on Windows, AX on macOS) exposes that tree
- A screen reader reads from the OS API — never from your DOM directly

This is why an ARIA attribute with no effect on layout can still completely change what a screen reader user experiences.

ARIA Semantics: Roles, States, Properties

Roles

- What an element IS
- role="dialog", role="tab"
- Prefer native HTML first

States

- Condition right now
- aria-expanded, aria-checked
- Must stay in sync

Properties

- Static relationships
- aria-label, aria-describedby
- Set once, rarely change

ARIA changes what's announced, never what's shown.

ARIA Pattern: Accessible Modal Dialog

components/ui/dialog-content.tsx

```
<div
  role="dialog"
  aria-modal="true"
  aria-labelledby="dialog-title"
  aria-describedby="dialog-desc"
>
  <h2 id="dialog-title">Delete this project?</h2>
  <p id="dialog-desc">This action cannot be undone.</p>

  <button onClick={onCancel}>Cancel</button>
  <button onClick={onConfirm} autoFocus>Delete</button>
</div>
```

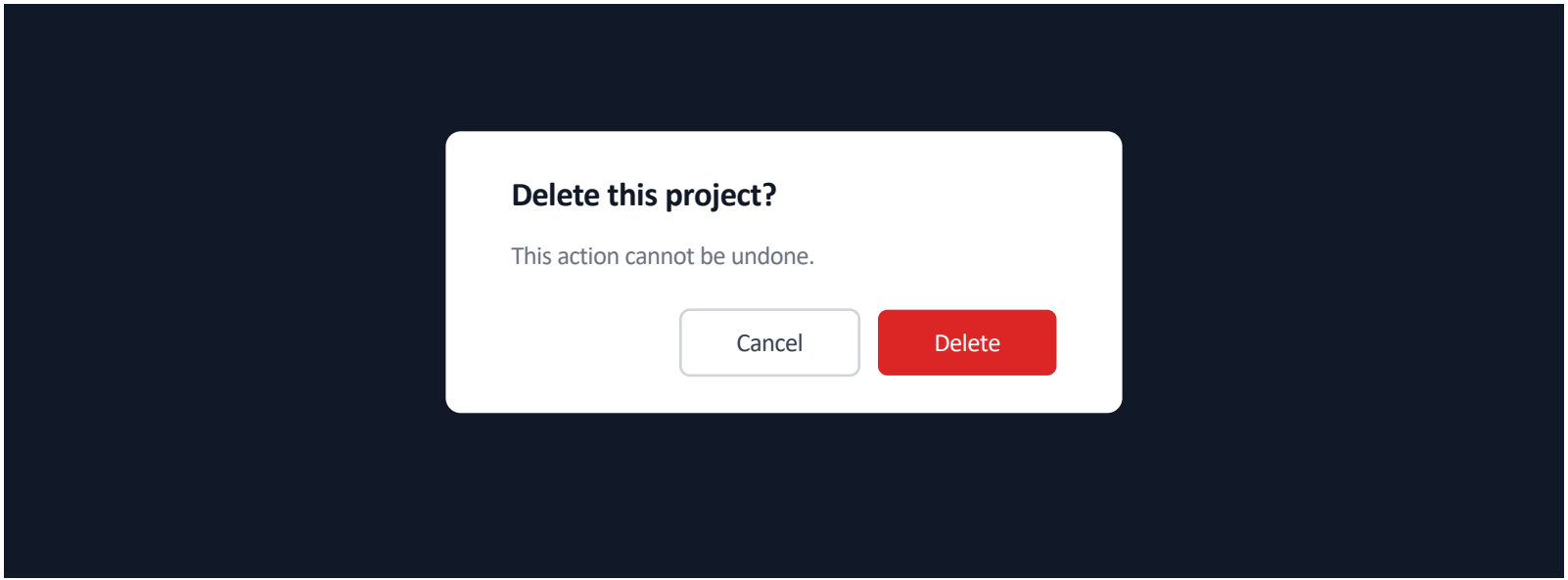
aria-modal

Marks content behind the dialog as inert.

labelledby / describedby

Links title and description for instant announcement.

What the Modal Looks Like, Open



The page behind is dimmed and inert — aria-modal is what tells a screen reader the same thing.

Semantic HTML First, ARIA Last

The first rule of ARIA: don't use it if native HTML already works.

Reach For

- <button> for anything clickable
- <nav>, <main>, <header>
- <label> on every input
- / for lists

Not This

-
-
- aria-label patched on later
-

> *Native elements give you keyboard behavior for free.*

Practical Considerations: Needs → Solutions

Every rule in this chapter maps back to a real barrier someone actually hits — here's the map.

Need	Barrier If Ignored	Solution
Blind / screen reader users	Can't perceive a purely visual UI	Semantic HTML + ARIA + alt text
Low vision / color blindness	Can't read or distinguish content	4.5:1 contrast, never color-only
Motor impairment	Can't use a mouse or precise gestures	Full keyboard support, large targets
Cognitive / learning differences	Complex UI overwhelms, times out	Plain language, consistent nav, no time pressure
Deaf / hard of hearing	Can't perceive audio content	Captions and transcripts

Nothing in this chapter is abstract — every rule traces back to one specific person being unable to complete a task.

AI and ARIA: A Rocky Relationship

LLMs have seen millions of ARIA examples — and learned the wrong lesson from many of them.

Under-Uses ARIA

- Custom dropdown with zero role/state wiring
- No aria-live region for async updates

Over-Uses ARIA

- role="button" bolted onto an actual <button>
- aria-label duplicating visible text unnecessarily

Both directions come from the same root cause: pattern-matching without an accessibility tree in mind. More in Chapter 4.

The Criteria Developers Hit Most

Criterion	Requirement	Level
1.4.3 Contrast	4.5:1 normal text, 3:1 large text	AA
2.1.1 Keyboard	All functionality via keyboard	A
2.4.7 Focus Visible	Focus indicator must be visible	AA
4.1.2 Name, Role, Value	Components expose name/role/value	A

Know these four cold.

Keyboard & ARIA Decision Guide

? Building a custom widget?	→ **Check WAI-ARIA patterns first**
? Mouse-only today?	→ **Still needs keyboard support**
? Native HTML does the job?	→ **Use it, skip ARIA**
? Purely decorative icon?	→ **aria-hidden="true"**

Chapter 2 Key Terms

Background material worth keeping.

Terms

- POUR — Perceivable, Operable, Understandable, Robust: WCAG's four principles
- WCAG AA — the legal/industry conformance target: 4.5:1 contrast, full keyboard support
- Focus trap — keeps Tab cycling inside an open widget, e.g. a modal
- ARIA — attributes that describe dynamic state to assistive technology
- Accessible name — the text a screen reader announces for an element

END OF CHAPTER 2

WCAG 2.1 AA

- ✓ AA is the target: 4.5:1 contrast, full keyboard support
- ✓ ARIA translates dynamic UI for assistive tech
- ✓ Semantic HTML first, always
- ✓ AI both under- and over-uses ARIA — verification catches both
- ✓ Five keys cover the entire keyboard bar

The takeaway: *Accessibility is a constraint that shapes architecture, not a checklist bolted on after.*

Today's Story, in Four Chapters

~~Component architecture — Tailwind CSS and shadcn/ui~~

~~WCAG 2.1 AA — keyboard navigation and ARIA semantics~~

Ch.3 axe-core — automated accessibility testing

Ch.4 AI-generated UI verification

Up next

axe-core — how to automate the parts of WCAG a machine can actually check, and know where it stops.

Chapter 2: Poll

CHAPTER 3

axe-core: Automated Accessibility Testing

Manual audits can't keep pace with fifty pull requests a week.

Where Manual-Only Audits Failed

The audit process this chapter replaces — and the case that made ignoring it expensive.

- 2006 — National Federation of the Blind sues Target Corp. over Target.com
- 2008 — Target settles for $6M; courts rule the ADA applies to websites too
- Manual audits still don't scale: one reviewer, one snapshot in time
- Different auditors flagged different issues for the same page
- By the next release, the report was already out of date

> *The Target case proved the legal stakes were real years before the tooling caught up to make prevention practical.*

How axe-core Came to Be

- For years, audits meant a specialist reviewing pages by hand
- By the time the report arrived, the code had moved on
- 2017 — Deque Systems open-sources axe-core
- Design rule: zero false positives, even if that means fewer catches
- Trust earned once made it safe to gate CI on

Never cry wolf — that's why it became the default.

How the Engine Works

What It Checks

- Color contrast
- ARIA validity
- Form labeling
- Heading structure

The Design Rule

- Zero false positives
- Scans the rendered DOM
- Framework-agnostic
- Under-reports on purpose

A clean scan actually means something.

What It Catches — and Doesn't

57%

of WCAG issues caught automatically

0

false positives, by design

43%

still need a human

A floor, not a ceiling.

Component Test with jest-axe

components/ui/button.test.tsx

```
import { render } from "@testing-library/react";
import { axe, toHaveNoViolations } from "jest-axe";
import { Button } from "./button";

expect.extend(toHaveNoViolations);

it("has no accessibility violations", async () => {
  const { container } = render(
    <Button variant="destructive">Delete project</Button>
  );
  const results = await axe(container);
  expect(results).toHaveNoViolations();
});
```

One assertion catches an entire class of bugs.

Can AI Write Your axe-core Tests?

Yes — with the same review discipline as any AI-generated test.

AI Is Good At

- Scaffolding the jest-axe boilerplate
- Covering the "happy path" render
- Writing the Playwright config

Still Check

- Does it test every variant, not just default?
- Did it assert on the right container?
- Does it actually run in CI, not just locally?

AI-written tests need the same Generate-Audit-Fix loop as AI-written components — Chapter 4 makes this explicit.

Full-Page Scan with Playwright + axe

e2e/dashboard.a11y.spec.ts

```
import { test, expect } from "@playwright/test";
import AxeBuilder from "@axe-core/playwright";

test("dashboard has no critical a11y issues", async ({ page }) => {
  await page.goto("/dashboard");
  const results = await new AxeBuilder({ page })
    .withTags(["wcag2a", "wcag2aa"])
    .analyze();

  const serious = results.violations.filter((v) =>
    ["critical", "serious"].includes(v.impact ?? "")
  );
  expect(serious).toEqual([]);
});
```

Fail the build on critical/serious only.

Reading Violation Impact Levels

Impact	Example	CI Action
Critical	Input with no accessible name	Block merge
Serious	Insufficient color contrast	Block merge
Moderate	Redundant ARIA role	Warn, track
Minor	Missing lang attribute	Warn, track

Calibrate the gate to impact, not raw count.

What Automated Testing Can Never Catch

Invisible to axe-core

- Does reading order make sense?
- Is alt text actually meaningful?
- Can the task be completed?

Needs a Human

- Real screen reader walkthrough
- Keyboard-only task completion
- Zoom to 200%, check for clipping

Clean scan = no mechanical defects, not "fully accessible."

Automated vs Manual — When Is Each Enough?

? Routine PR on existing components?	→ axe-core in CI is enough
? New critical flow (checkout)?	→ Add a manual screen-reader pass
? Fully custom widget?	→ Manual testing required

Chapter 3 Key Terms

Background material worth keeping.

Terms

- axe-core — the open-source, zero-false-positive accessibility rules engine
- jest-axe — runs axe-core inside a component unit test
- Impact level — critical, serious, moderate, minor; drives the CI gate
- CI gate — a check that blocks a merge on critical/serious violations
- False positive — a flagged issue that isn't actually a real violation (axe-core has none, by design)

END OF CHAPTER 3

axe-core Automated Testing

- ✓ axe-core: zero false positives, ~57% of issues caught
- ✓ Layer it: dev warnings, unit tests, CI scans
- ✓ It's a floor, not a ceiling
- ✓ AI can draft the tests — you still review coverage
- ✓ Gate CI on impact level, not raw violation count

The takeaway: *Automate the floor, so your human review time goes toward the ceiling.*

Today's Story, in Four Chapters

- ~~Component architecture — Tailwind CSS and shadcn/ui~~
- ~~WCAG 2.1 AA — keyboard navigation and ARIA semantics~~
- ~~axe-core — automated accessibility testing~~
- **Ch.4** AI-generated UI verification

Up next

AI-generated UI verification — why an AI that writes inaccessible code was never asked to imagine a screen reader.

Chapter 3: Poll

CHAPTER 4

AI-Generated UI Verification

axe-core doesn't care who wrote the code. That turns out to be exactly the problem.

Why This Chapter Exists

Picking up directly from Chapter 3 — axe-core still works. What's changed is the volume of code hitting it.

- axe-core (Chapter 3) scans any rendered DOM — it doesn't know or care who wrote the markup
- 2022-today — AI agents now write a growing share of that markup
- The rules haven't changed; the volume and speed of code produced has
- Most training code was visually correct, rarely screen-reader tested
- Not malice — accessibility was never the training signal

> *axe-core is still the right tool — Chapter 4 is about the workflow discipline that makes sure it actually gets run.*

Common AI Accessibility Failure Patterns

Structural

- <div onClick> not <button>
- Icon buttons with no label
- Headings skip levels

Interactive

- Dropdowns with no keyboard support
- Modals that don't trap focus
- Color as the only signal

These four patterns cover most violations we see.

Before: A Typical AI-Generated Component

components/status-badge.tsx (as generated)

```tsx
function StatusBadge({ status }) {
  const color = status === "error" ? "red" : "green";
  return (
    <div
      className={`badge ${color}`}
      onClick={() => onDismiss(status)}
    >
      {status === "error" ? "!" : "OK"}
    </div>
  );
}
```

Violations

No keyboard access, color-only meaning, no accessible name.

Looks right in a screenshot. That's the danger.

After: Verified and Fixed

components/status-badge.tsx (corrected)

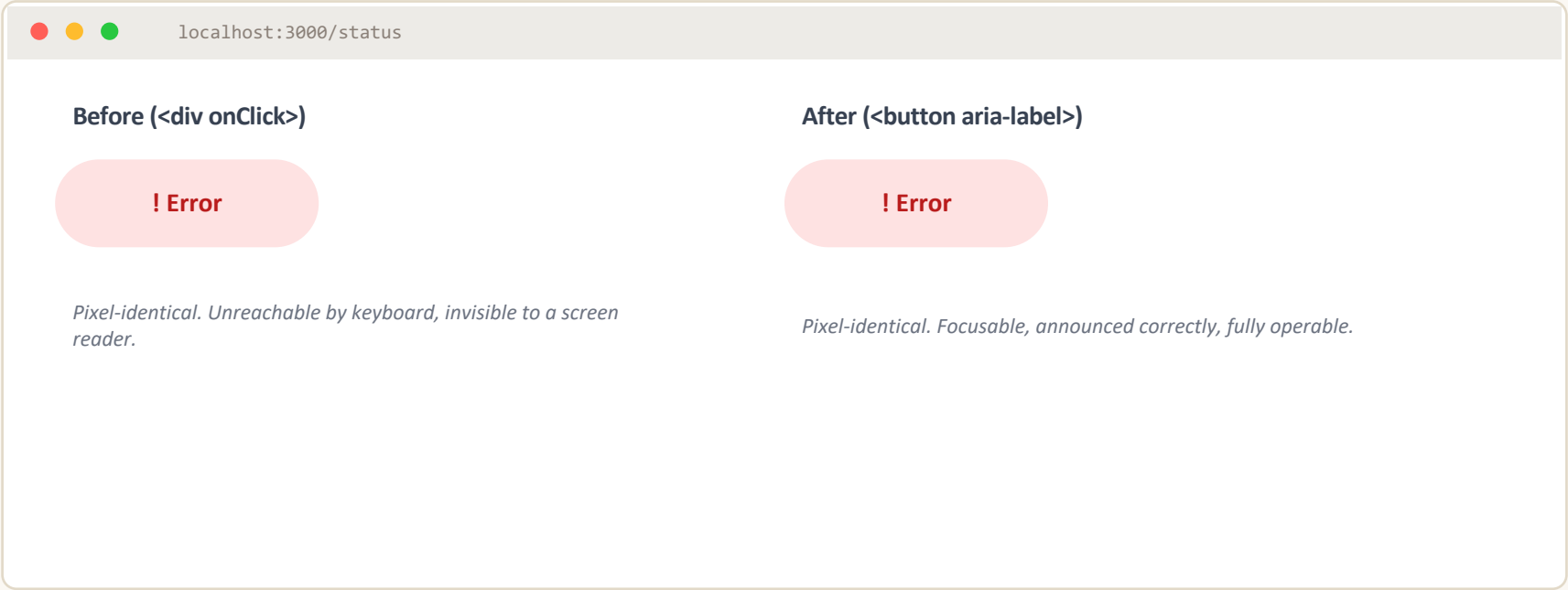
```tsx
function StatusBadge({ status }: { status: "error" | "ok" }) {
  const isError = status === "error";
  return (
    <button
      type="button"
      onClick={() => onDismiss(status)}
      aria-label={`Dismiss ${status} status`}
    >
      <span aria-hidden="true">{isError ? "!" : "OK"}</span>
      <span className="sr-only">{isError ? "Error" : "OK"}</span>
    </button>
  );
}
```

Fixed

Native button, real label, keyboard-operable.

Both Versions Look Completely Identical

This is the entire point — the visual output can't tell you which one is broken.



You cannot code-review this by looking at a screenshot — you have to check the markup, or run axe-core.

The Generate, Audit, Fix, Re-Test Loop

1. Generate

- AI produces a first draft

2. Audit

- Run axe-core immediately

3. Fix

- Prefer semantic HTML

4. Re-Test

- Confirm it's actually clean

Minutes now, versus an incident later.

Prompting for Accessibility

Weak Prompt

- "Build a dropdown menu"
- No a11y requirement stated
- Model defaults to div-based

Strong Prompt

- "Follow WAI-ARIA Menu pattern"
- "Fully keyboard-operable"
- "I'll run axe-core after"

State it in the spec, like a type requirement.

A Two-Minute Verification Checklist

Automated

- axe-core: zero critical/serious
- Strict TypeScript passes

Manual (60 sec)

- Tab through every element
- Enter/Space triggers actions
- Zoom to 200%, check clipping

Catches nearly everything on the failure list.

Acceptance Criteria for AI-Generated UI

? Any critical/serious axe violation?	→ **Reject, fix first**
? used as a clickable element?	→ **Reject, require semantic HTML**
? Clean scan, no manual pass yet?	→ **Do the 60-second pass**
? Both pass?	→ **Approve**

WORTH REMEMBERING

"An AI that writes inaccessible code was never asked to imagine a screen reader. That imagination has to come from you."

Chapter 4 Key Terms

Background material worth keeping.

Terms

- Generate-Audit-Fix-Retest — the standard loop for any AI-generated UI
- Accessibility tree — the structure assistive tech actually reads, separate from the visual DOM
- Spec clarity — stating requirements (incl. accessibility) explicitly in a prompt
- sr-only — a CSS class that hides text visually but keeps it for screen readers
- Acceptance criteria — the pass/fail bar a component must clear before merge

END OF CHAPTER 4

AI-Generated UI Verification

- ✓ AI optimizes for visual correctness, not accessibility
- ✓ Generate, Audit, Fix, Re-test — every component
- ✓ Never trust, always verify
- ✓ Identical screenshots can hide very different accessibility
- ✓ State accessibility requirements directly in the prompt

The takeaway: *Never trust, always verify — the entire chapter in five words.*

What we have seen today

- ~~Component architecture — Tailwind CSS and shadcn/ui~~
- ~~WCAG 2.1 AA — keyboard navigation and ARIA semantics~~
- ~~axe-core — automated accessibility testing~~
- ~~AI-generated UI verification~~

Up next

The hands-on lab — build the Generate, Audit, Fix, Re-test loop yourself, on a real modal.

Chapter 4: Poll

CHAPTER Lab

Hands-On: An AI-Generated Modal, Audited and Fixed

Thirty minutes, one component, the full loop — on your own machine.